

Práctico: Pilas y Colas con Arreglos en Java usando OpenCode

Objetivo general

El objetivo de este práctico es que implementen estructuras de datos utilizando Java y OpenCode, pero no copiando código directamente.

Antes de pedirle a OpenCode que genere la solución, deberán escribir un prompt claro donde expliquen:

qué estructura necesitan, qué operaciones debe tener, qué casos límite deben contemplarse, cómo debe manejarse el arreglo interno y qué comportamiento esperan obtener.

La calidad del código generado dependerá directamente de la calidad del razonamiento planteado en el prompt.

Ejercicio 1: Implementar una pila de enteros

Implementar una clase `PilaEnteros` utilizando un arreglo de tamaño fijo.

Debe permitir:

`push`, `pop`, `peek`, `isEmpty`, `isFull` y `size`.

El alumno debe explicar en el prompt cómo se usa el índice `top`, qué valor inicial debe tener y qué sucede cuando la pila está llena o vacía.

Ejercicio 2: Simulador de torre de platos

Implementar un programa que simule una torre de platos.

Cada plato puede representarse con un número o nombre.

El programa debe permitir:

agregar un plato, retirar el plato superior y consultar cuál es el plato que está arriba.

El prompt debe explicar por qué este problema se resuelve con una pila y no con una cola.

Ejercicio 3: Validar paréntesis balanceados

Implementar un programa que reciba una expresión matemática y determine si los paréntesis están correctamente balanceados.

Ejemplo:

$(5 + 3) * (2 + 1)$

Debe devolver válido.

Ejemplo:

$(5 + 3)) * (2 + 1)$

Debe devolver inválido.

El prompt debe explicar cómo se usa una pila para guardar paréntesis abiertos y cuándo se realiza un **pop**.

Ejercicio 4: Historial de navegación

Implementar un simulador simple de historial de navegación.

Cada vez que el usuario visita una página, se apila la URL.

El programa debe permitir volver a la página anterior usando **pop**.

El prompt debe explicar por qué el historial funciona bajo el principio LIFO.

Ejercicio 5: Implementar una pila genérica

Implementar una clase `Pila<T>` usando arreglos.

Debe permitir almacenar distintos tipos de datos:

`Integer`, `String` u objetos simples.

El prompt debe explicar qué problema resuelve el uso de genéricos y cómo se diferencia de una pila hecha solamente con `int`.

Ejercicio 6: Implementar una cola simple de enteros

Implementar una clase `ColaEnteros` usando un arreglo.

Debe permitir:

`enqueue`, `dequeue`, `front`, `isEmpty`, `isFull` y `size`.

El prompt debe explicar para qué sirven los índices `front` y `rear`, y cómo cambian cuando se agregan o eliminan elementos.

Ejercicio 7: Sistema de turnos para atención

Implementar un sistema de turnos para una oficina.

Cada persona que llega recibe un turno y se agrega a una cola.

El sistema debe permitir:

agregar persona, atender siguiente persona y consultar quién está primero.

El prompt debe explicar por qué este problema utiliza FIFO y no LIFO.

Ejercicio 8: Cola de impresión

Implementar un sistema de cola de impresión.

Cada documento tiene un nombre y una cantidad de páginas.

Los documentos se imprimen en el mismo orden en que fueron agregados.

El prompt debe explicar cómo una cola representa correctamente el orden de impresión.

Ejercicio 9: Problema de desperdicio de espacio en cola simple

Implementar una cola simple con arreglo y demostrar el problema del desperdicio de posiciones.

El programa debe mostrar el arreglo luego de varias operaciones **enqueue** y **dequeue**.

El prompt debe pedir explícitamente que se vea cómo quedan espacios libres al inicio, pero no pueden reutilizarse en una cola simple.

Ejercicio 10: Cola circular

Implementar una cola circular utilizando arreglo.

Debe reutilizar los espacios liberados al inicio del arreglo.

El prompt debe explicar cómo se usa el operador módulo **%** para mover **front** y **rear**, y cómo se detecta si la cola está llena o vacía.

Cierre del práctico

En cada ejercicio, el alumno debe entregar:

1. El código Java generado.
2. Un comentario al inicio del archivo con el prompt inicial usado en OpenCode.
3. Una breve explicación de si tuvo que ajustar el prompt.

4. Una prueba de ejecución con datos de ejemplo.